



QualiTrace

Pilotage de la conformité et traçabilité des flux industriels.

Conception | Jérémy MOREAU – 2025-2026

Version	Date	Auteur	Objet
1.0	23 juin 2026	Jérémy MOREAU	Finalisation du document initial
0.1	25 mai 2026	Jérémy MOREAU	Création du document « Conception »

Crédits Graphiques : Logo de « QualiTrace » généré par l'IA Gemini (Modèle Nano Banana), Google, 2026.

Table des matières

1. Objectif du document.....	3
2. Architecture Logicielle et Déploiement.....	4
2.1. Contraintes techniques.....	4
2.2. Architecture globale.....	5
2.3. Schéma d'architecture applicative.....	7
2.4. Architecture de déploiement.....	8
2.5. Schéma de base de données.....	10
3. Choix Technologiques.....	11
3.1. Back-end.....	11
3.2. Persistance des données.....	11
3.3. Front-end.....	11
3.4. Serveur web.....	12
3.5. Monitoring.....	12
3.6. Environnements de développement.....	12
3.7. Tests & Qualité logicielle.....	13
3.8. Sécurité et gestion des sessions.....	13
4. Conception Détaillée par Cas d'Utilisation.....	14
4.1. UC-UTI-01 – S'identifier.....	14
4.2. UC-UTI-03 – Gérer les comptes utilisateurs.....	15
4.3. UC-REF-04 – Paramétrer les gammes de contrôles.....	16
4.4. Gestion des lots.....	17
4.5. Autres endpoints.....	18
5. Regroupement des Classes (API / Back-end).....	19
Annexes.....	20
I. Terminologie.....	20

1. Objectif du document

Ce document présente l'**architecture** et la **conception** du système **QualiTrace** pour le compte de la société **BadChemicals**, selon la méthode Arrington. Il s'appuie sur l'**Expression des Besoins** et sur l'**analyse** étudiées précédemment, dont il constitue le prolongement technique.

L'objectif principal de ce document est de **traduire l'architecture logique** et les **objets du domaine** établie lors de la phase d'analyse **en une solution technique concrète, robuste et directement implémentable**. Il vise à spécifier la structure des composants logiciels, le Modèle Physique de Données (MPD), et les mécanismes transverses, tels que la traçabilité et l'infrastructure cible, afin de lever toute ambiguïté technique avant la phase de développement.

Ce document **se concentre exclusivement** sur les **éléments** applicatifs et cas d'utilisation **critiques** identifiés lors **de l'analyse**. Les aspects standards ou natifs du framework ne sont pas détaillés afin de concentrer l'effort de conception sur les exigences d'intégrité du cœur de métier.

Dans ce document, nous commencerons par rappeler les **contraintes techniques du projet** et nous étudierons l'**architecture du logiciel** et les **technologies choisies** pour le projet ainsi que l'**infrastructure de déploiement cible**. Les choix effectués ne sont cependant pas définitifs et l'objectif reste avant tout la **fiabilité** et la **scalabilité** du projet.

Ce document servira de référence pour l'équipe de développement lors de la phase de développement.

2. Architecture Logicielle et Déploiement

2.1. Contraintes techniques

QualiTrace est un outil à destination des industries cosmétiques et pharmaceutiques, un domaine qui est encadré de façon stricte par les **Bonnes Pratiques de Fabrication** (BPF) et la norme **21 CFR Part 11** (relative aux enregistrements et signatures électroniques).

Pour garantir la conformité du système, **QualiTrace** doit impérativement répondre à des exigences de :

- **Sécurité et Confidentialité** : Chaque équipe a un rôle bien défini. **Le système devra être sécurisé** et faire en sorte que chaque personne dispose de son propre accès et ne puisse effectuer que les actions autorisées par son profil. L'accès au système et aux données devra être strictement restreint aux utilisateurs authentifiés et autorisés.
 - Cela implique une **gestion fine des droits d'accès**, un **contrôle des droits** sur chaque page et la **possibilité de se déconnecter instantanément**.
- **Traçabilité et Audit Trail** : Les normes pharmaceutiques impliquent une **traçabilité complète** de toutes les actions effectuées (audit trail). Toute action effectuée sur le système devra donc être tracée de façon immuable permettant ainsi de reconstituer l'historique des événements (Qui, Quand, Quoi, Pourquoi).
 - Cela implique l'**enregistrement de toutes les informations d'ajout ou de modification** de manière **immuable**, mais également **impossibilité technique de supprimer physiquement des données**.
- **Disponibilité et Intégrité des données** : Le système devra garantir que les données ne sont pas altérées, accidentellement ou non, tout au long de leur cycle de vie. Il devra permettre, aux personnes disposant des droits nécessaires, **d'éditer les documents légaux** (Certificat d'analyse) et **fournir les informations** nécessaires en cas d'investigation lors de déviations, rappels de lots, audits/inspections ou tout besoin similaire.
 - Cela implique une **base de données robuste**, avec des **relations fortes** et des **transactions sûres** (propriétés ACID).

2.2. Architecture globale

Le système **QualiTrace** adopte une architecture moderne entièrement découplée, séparant la logique métier et la persistance (*Back-end*) de l'interface utilisateur et de l'expérience interactive (*Front-end*). Ce choix d'architecture s'articule autour des principes d'une **Single Page Application** (SPA) communiquant via une **API RESTful**, structurée selon les patterns du **Domain-Driven Design** (DDD).

2.2.1. Une architecture découplée API RESTful / SPA

Plusieurs options d'architectures ont été étudiées et confrontées :

- **Architecture monolithique** (Ex. Rendu côté serveur avec *Thymeleaf*) : Bien que plus simple à mettre en œuvre au départ, elle conduit inévitablement à une liaison forte entre la logique métier et la présentation, entraînant un couplage fort et proposant des performances généralement moins bonnes en raison du rechargement complet des pages à chaque changement. Cette architecture peut rapidement devenir difficile à maintenir avec le temps et les évolutions du logiciel.
- **Architecture hybride** monolithique + front moderne (Ex. *Thymeleaf* + *HTMX*) : Elle apporte une amélioration certaine quant à la dynamique des pages et reste relativement simple à mettre en œuvre. Toutefois, cette architecture conserve un couplage fort et limite l'ouverture du système aux futurs besoins évoqués de liaison avec d'autres logiciels (ERP, MES et scannettes).
- **Architecture découplée** (SPA + API REST) : Elle implique le développement du logiciel en deux parties indépendantes : la logique métier et la persistance (*Back-end*) d'une part, et l'interface utilisateur (*Front-end*) d'autre part. Bien qu'initialement plus complexe, elle offre plusieurs avantages :
 - **Évolutivité et interopérabilité** : l'exposition d'une API REST permet l'interfaçage avec d'autres logiciels ou terminaux de saisie déportée et une évolution de la logique métier ou de l'interface utilisateur de manière totalement séparée.
 - **Séparation des responsabilités** : Le développement, les tests et le déploiement des composants Front et Back deviennent indépendants.

- **Performance et fluidité** : Une SPA ne recharge que les données nécessaires et offre une réactivité optimale. L'expérience utilisateur en est grandement améliorée, particulièrement sur un logiciel de laboratoire utilisé au quotidien comme **QualiTrace**.

C'est donc l'**architecture découplée**, SPA + API REST, qui sera choisie.

2.2.2. Structure du Back-end : Domain-Driven Design

QualiTrace évolue dans un **domaine métier très complexe** et où toute **ambiguïté** ou **mauvaise compréhension** entraîne des **risques majeurs**.

Pour faciliter les échanges entre les équipes métiers et les équipes techniques, le DDD est l'approche idéale. Le code sera organisé par domaine métier (**Domain**) et la logique sera développée de manière totalement indépendante des frameworks et de la couche technique (**Infrastructure**). La couche applicative (**Application**) orchestre l'ensemble et gère les cas d'utilisation.

2.2.3. Traçabilité et contrôle du code source

La traçabilité est un prérequis indispensable à tous les niveaux pour un logiciel comme **QualiTrace**. La traçabilité du code source et des changements se fera via l'outil de contrôle de version distribué **Git** et la plateforme **GitHub**. Un contrôle automatisé du code source et des configurations sera réalisé via la CI (Continuous Integration) de GitHub : les **GitHub Actions**.

Un dépôt GitHub a été spécialement créé pour le projet :

<https://github.com/neeftarah/qualitrace>

2.3. Schéma d'architecture applicative

Transition Arrington ⇒ Spring Boot

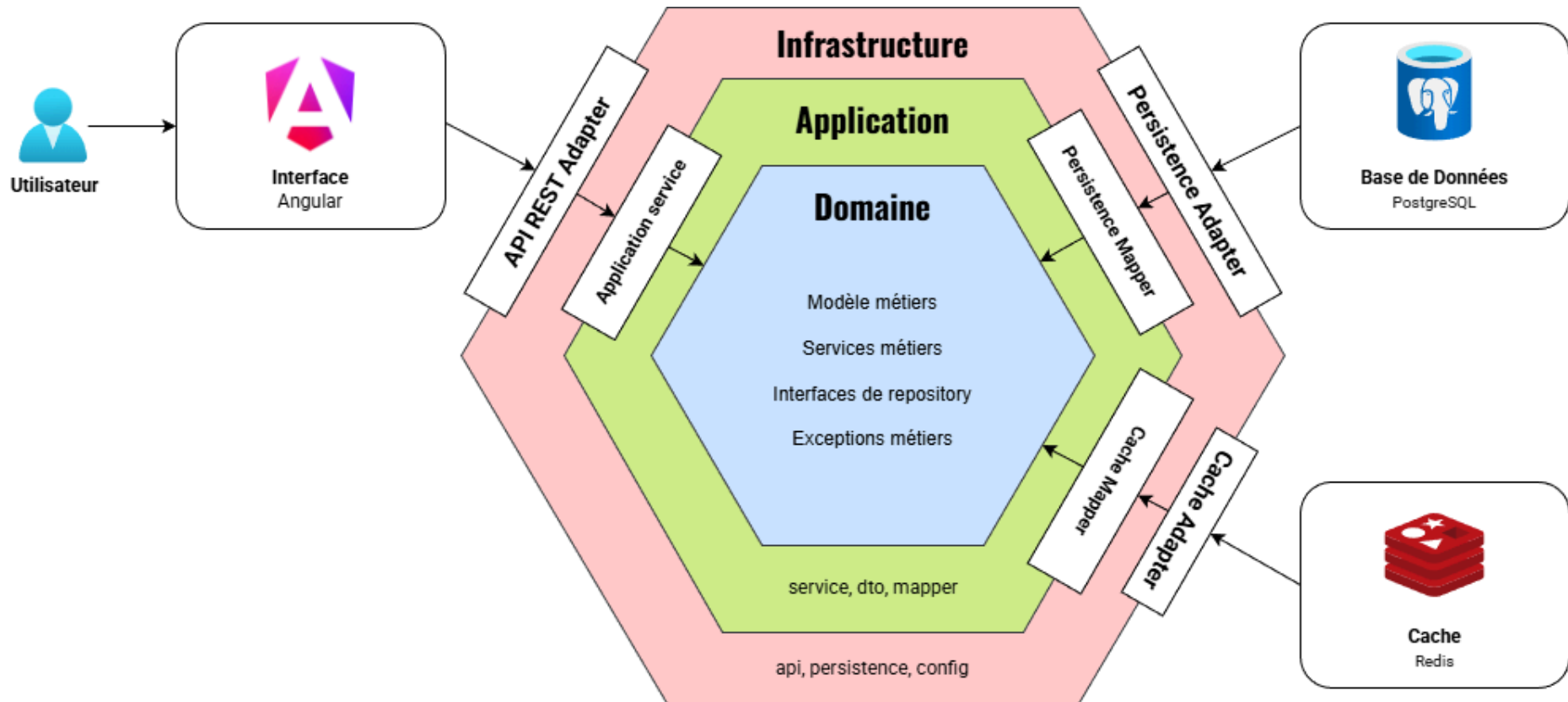
Afin de passer de l'analyse Arrington à une architecture technique Spring Boot, nous pouvons appliquer les patterns suivants :

- **Boundary** (IHM) ⇒ **Infrastructure / Presentation** :
Composants **Angular** + PrimeNG
- **Boundary** (API) ⇒ **Infrastructure / Web** :
Contrôleurs REST (@RestController)
- **Control** (Workflow/Logique) ⇒ **Application / UseCases** :
Services applicatifs (@Service Spring, sans logique métier)
- **Entity** (Données métiers) ⇒ **Domain / Model** :
Objets **JAVA pures** (Entités, Value objects)
- **LifeCycle** (Persistance/DAO) ⇒
 - **Domain / Repository** → Interfaces de l'architecture (Contrats)
 - **Infrastructure / Repository** → Implémentation JPA / Hibernate

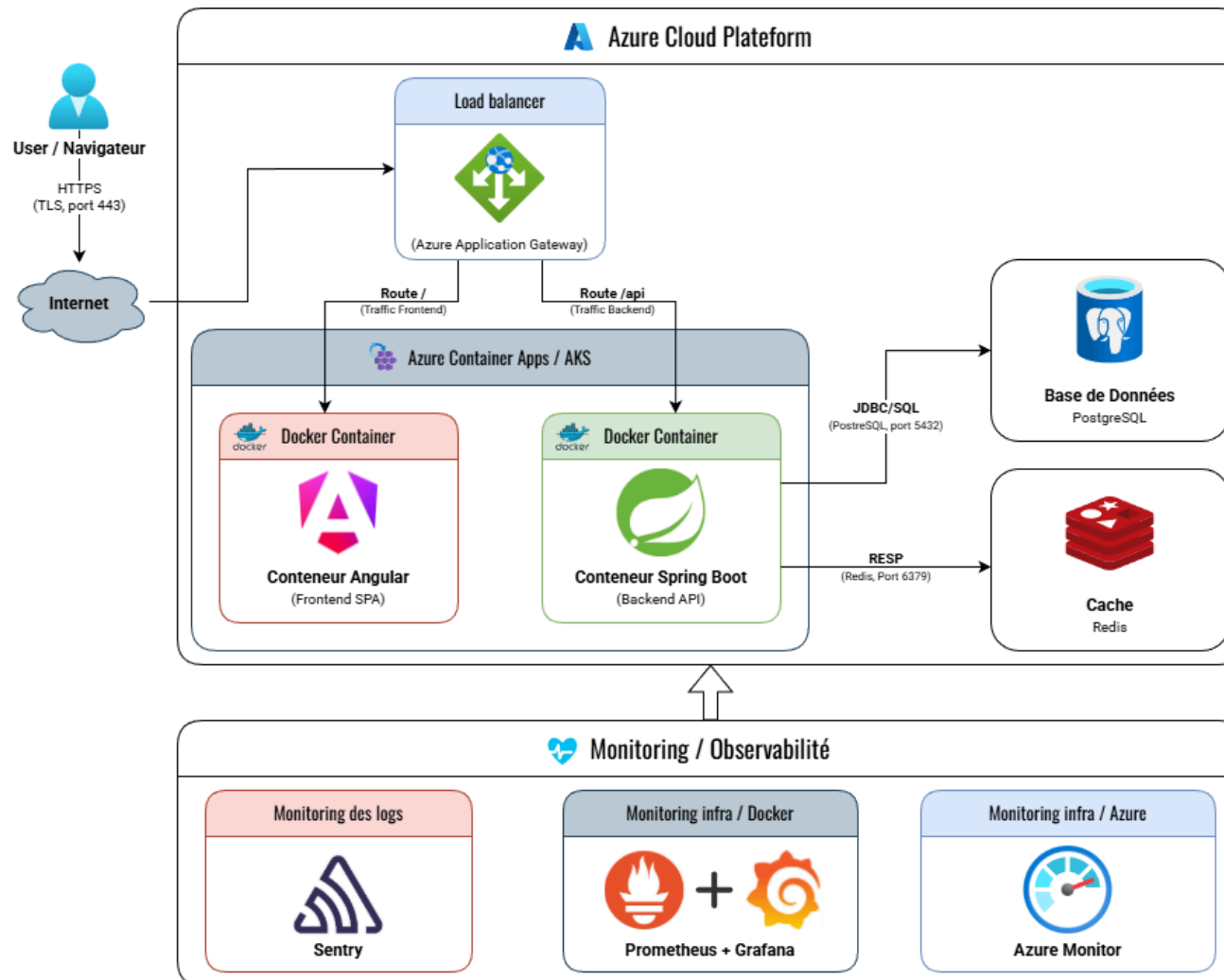
Définition des packages

- **domain** (Le Cœur métier - Objets "Entity" Arrington)
 - **model** : Les Entités, Value Objects, Agrégats et règles métier.
 - **repository** : Les interfaces de contrats.
 - **service** : Services métiers multi-entités (si nécessaire).
 - **exception** : Les exceptions "métier".
- **application** (L'Orchestrateur - Objets "Control" Arrington)
 - **service** : Les Services Applicatifs orchestre les entités du domaine.
 - **dto** : Les structures de données exposées à l'IHM Angular.
 - **mapper** : Les interfaces de conversion (ex: Convertir Domaine -> DTO).
- **infrastructure** : (Les Détails Techniques & les "Adaptateurs")
 - **api** : Les points d'entrée des API REST (Les @RestController Spring Boot)
 - **persistance** : (La persistance - Implémentation des "LifeCycle" Arrington)
 - **entities** : Les Entités JPA qui représentent les tables de la BDD.
 - **repositories** : Les implémentations concrètes des interfaces du domaine
 - **db** : Les Listeners JPA pour l'Audit Trail.
 - **config** : La configuration de l'application

Schéma d'architecture hexagonale / DDD de QualiTrace



2.4. Architecture de déploiement



2.5. Schéma de base de données



3. Choix Technologiques

3.1. Back-end

La qualité, la traçabilité et la sécurité imposées par **QualiTrace** suggèrent un environnement strict, orienté objet et fortement typé. Le langage **JAVA**, dans sa **version 25**, associé au framework **Spring Boot 4**, sont idéaux pour cette typologie de projet et reste simple et efficace à mettre en œuvre pour une application web moderne.

JAVA et **Spring Boot** s'adaptent parfaitement à la criticité du projet et au choix d'architecture DDD et API RESTful.

3.2. Persistance des données

La criticité des données de QualiTrace impose un système de stockage sûr et transactionnel (propriétés ACID).

PostgreSQL est une référence de l'industrie pour sa robustesse et son respect des normes SQL. Il offre toutes les fonctionnalités requises pour un logiciel comme QualiTrace tout en restant relativement léger, performant et open-source.

Spring Data JPA (Hibernate) sera également utilisé afin de simplifier les développements et assurer une meilleure corrélation entre entité logicielle et table de base de données.

Une base de données **Redis** sera également utilisée pour le cache applicatif, le stockage des sessions et du rate limiting.

L'historisation et la traçabilité des modifications (*Audit Trail*) seront assurées au niveau de la couche de persistance via des **EntityListeners JPA**, permettant un contrôle précis et sur mesure des données auditées.

3.3. Front-end

Pour créer une interface utilisateur moderne pour **QualiTrace**, une Single Page Application (SPA) sera développée via le framework **Angular 21**, couplée à la bibliothèque de

composants **PrimeNG** et le template de base **Sakai**. Cette composition offrira une interface moderne, performante et rapide à mettre en œuvre.

Deux autres frameworks JavaScript majeurs sont également fréquemment utilisés : React et VueJS. Toutefois **Angular** dispose d'une approche similaire à celle de JAVA et est plus adapté à une application critique comme **QualiTrace**.

3.4. Serveur web

Le projet étant développé avec JAVA et Spring Boot, nous utiliserons le serveur applicatif embarqué, **Tomcat**, pour servir l'application. C'est le standard de l'industrie (Cloud Native).

3.5. Monitoring

Le monitoring d'une application comme **QualiTrace** est indispensable afin de s'assurer de son bon fonctionnement sur la durée et anticiper les problèmes de charge.

Nous utiliserons **Spring Boot Actuator** pour la remontée de données sur l'application, **Sentry** pour la gestion des logs applicatifs, **Prometheus** et **Grafana** pour le monitoring des serveurs.

3.6. Environnements de développement

Le choix des environnements de développement, de test et de production est critique, particulièrement pour un projet critique comme **QualiTrace**. La reproductibilité est une exigence majeure.

La conteneurisation via **Docker** pour les environnements de développement et de test, offre une excellente reproductibilité et une simplicité d'utilisation idéale tandis que l'orchestration par **Kubernetes** en production, offre une excellente isolation, une haute disponibilité et une scalabilité adaptées aux exigences de l'industrie.

Ce choix garantit un environnement strictement iso-production, éliminant les dérives de configuration entre les postes de développement, le pipeline de **CI/CD** et l'infrastructure de déploiement cible.

3.7. Tests & Qualité logicielle

La fiabilité de **QualiTrace** est garantie par une pyramide de tests automatisés (**JUnit/Mockito** pour le *Domaine*, **MockMvc** pour l'*API*, **Jasmine/Karma** pour *Angular* et **Cypress** pour les flux *E2E*).

L'intégrité architecturale DDD et la propreté du code sont contrôlées de manière non bloquante via **ArchUnit**, **ESLint**, **SpotBugs** et une *Quality Gate* ciblée sur **SonarCloud**.

3.8. Sécurité et gestion des sessions

Pour répondre aux contraintes de sécurité sans subir les inconvénients des jetons JWT (*révocation complexe, informations en clair*), le système s'appuiera sur des jetons opaques gérés côté serveur via **Spring Session Data Redis**.

L'**authentification** reste ainsi **centralisée, révocable** immédiatement et **opaque** pour le client.

4. Conception Détaillée par Cas d'Utilisation

4.1. UC-UTI-01 – S'identifier

Code – Nom	UC-UTI-01 – S'identifier
Description	Authentification d'un utilisateur
Endpoint API	POST /api/v1/login
Séquence / Fichiers	1. <code>/infrastructure/api/AuthController.java</code> Point d'entrée du formulaire d'authentification (Login/Password). 2. <code>/infrastructure/config/SecurityConfig.java</code> Configure la chaîne de filtres Spring Security, intercepte et valide la route. 3. <code>/infrastructure/security/AuthenticationManager.java</code> Composant de Spring Security chargé d'orchestrer l'authentification. 4. <code>/infrastructure/security/UserDetailsServiceImpl.java</code> Charge l'utilisateur pour Spring Security et vérifie le mot de passe. 5. <code>/infrastructure/persistence/repositories/UserRepository.java</code> Interface/contrat de persistance pour récupérer l'utilisateur par son login. 6. <code>/infrastructure/persistence/entities/UserEntity.java</code> Entité JPA contenant le mot de passe haché et les rôles. 7. Table de BDD <code>user</code> 8. <code>/infrastructure/security/TokenProvider.java</code> Génère un token aléatoire et unique (Bearer). 9. <code>/infrastructure/cache/RedisSessionStore</code> Stockage en cache Redis du token et des infos utilisateur 10. <code>/infrastructure/persistence/db/LoginAttempt.java</code> Journalisation technique de la tentative de connexion à des fins de sécurité.

4.2. UC-UTI-03 – Gérer les comptes utilisateurs

Code – Nom	UC-UTI-03 – Gérer les comptes utilisateurs		
Description	Permet à un administrateur de rechercher, créer, modifier ou désactiver un compte utilisateur.		
Endpoint API de sélections	GET	/api/v1/users	// Récupère la liste des utilisateurs
	POST	/api/v1/users/search	// Recherche des utilisateurs
	GET	/api/v1/users/{id}	// Récupère les infos d'un utilisateur
Séquence / Fichiers de sélections	1. /infrastructure/api/UserController.java Points d'entrée de la requête 2. /application/service/UserService.java Orchestre la récupération et la validation des données 3. /domain/repository/UserRepository.java Interface / contrat de persistance des données 4. /application/mapper/UserMapper.java Mapper de conversion de l'objet User métier vers JPA et inversement 5. /infrastructure/persistence/entities/User.java Entité JPA "User" 6. /infrastructure/persistence/repositories/UserRepository.java Implémentation JPA de la persistance des données 7. Table de BDD user		
Endpoint API d'ajouts / modifications	POST	/api/v1/users	// Crée un nouvel utilisateur
	PUT	/api/v1/users/{id}	// Modifie un utilisateur
	DELETE	/api/v1/users/{id}	// Désactive un utilisateur
Séquence / Fichiers d'ajouts / modifications	1. /infrastructure/api/UserController.java Points d'entrée de la requête 2. /application/service/UserService.java Orchestre la récupération et la validation des données 3. /domain/model/User.java Entité "User" et règles métier de validation des informations 4. /domain/repository/UserRepository.java Interface / contrat de persistance des données 5. /application/mapper/UserMapper.java Mapper de conversion de l'objet User métier vers JPA et inversement 6. /infrastructure/persistence/entities/User.java Entité JPA "User" 7. /infrastructure/persistence/repositories/UserRepository.java Implémentation JPA de la persistance des données 8. /infrastructure/persistence/db/AuditTrail.java Traçabilité des modifications 9. Table de BDD user		

4.3. UC-REF-04 – Paramétrer les gammes de contrôles

Code – Nom	UC-REF-04 – Paramétrer les gammes de contrôles
Description	Permet à un administrateur de définir les analyses et les seuils de conformité pour un article.
Endpoint API	GET /api/v1/components/{componentId}/controls // Récupère les gammes de contrôles d'un composant POST /api/v1/components/{componentId}/controls // Crée une nouvelle les gammes de contrôles PUT /api/v1/components/{componentId}/controls/{controlId} // Modifie une gamme de contrôles DELETE /api/v1/components/{componentId}/controls/{controlId} // Désactive une gamme de contrôles
Séquence / Fichiers	1. /infrastructure/api/ControlRangeSpecificationController.java Points d'entrée de la requête 2. /application/service/ControlRangeSpecificationService.java Orchestre la récupération et la validation des données 3. /domain/model/ControlRangeSpecification.java * Entité "ControlRangeSpecification" et règles métier 4. /domain/repository/ControlRangeSpecificationRepository.java Interface / contrat de persistance des données 5. /application/mapper/ControlRangeSpecificationMapper.java Mapper de conversion de l'objet métier vers JPA et inversement 6. /infrastructure/persistence/entities/ControlRangeSpecification.java Entité JPA "ControlRangeSpecification" 7. /infrastructure/persistence/repositories/ControlRangeSpecificationRepository.java Implémentation JPA de la persistance des données 8. /infrastructure/persistence/db/AuditTrail.java * Traçabilité des modifications 9. Table de BDD control_range_specification

* Les étapes 3 (validation des données) et 8 (Audit Trail) ne sont pas utilisées pour le endpoint GET car il ne modifie rien.

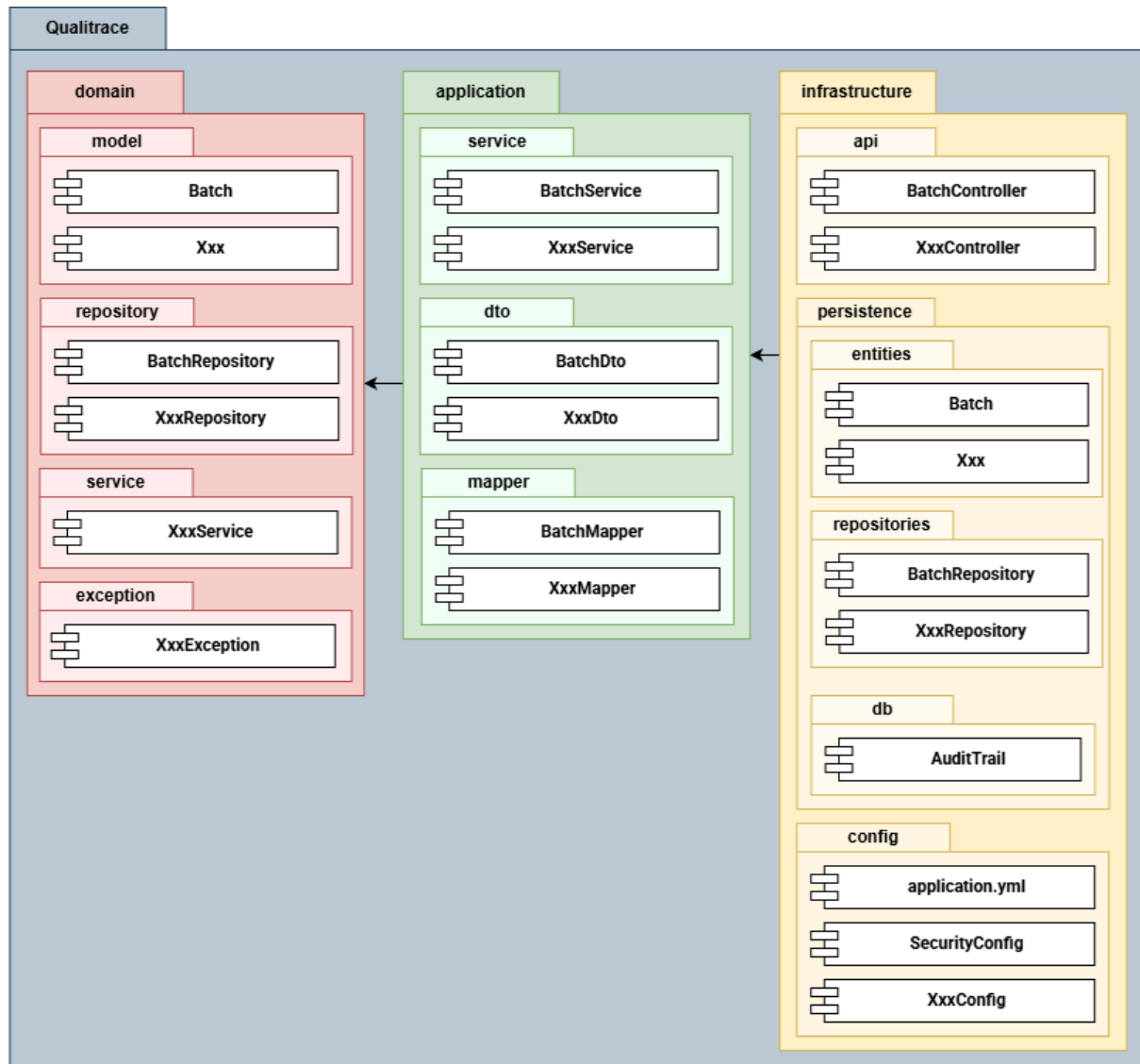
4.4. Gestion des lots

Code – Nom	UC-STK-02 – Réceptionner et étiqueter les lots UC-QUA-02 : Évaluer la conformité d'un lot
Description	Vérifie les pré-requis et contraintes et crée un lot ou valide sa conformité
Endpoint API	POST /api/v1/batches PUT /api/v1/batches/{id}/validate
Séquence / Fichiers	1. /infrastructure/api/BatchController.java Points d'entrée de la demande de validation 2. /application/service/BatchService.java Orchestre la validation de lot (Entité métier, persistance, audit trail, etc.) 3. /domain/model/Batch.java Entité "Batch" et règles métier de validation 4. /domain/repository/BatchRepository.java Interface / contrat de persistance des données 5. /application/mapper/BatchMapper.java Mapper de conversion de l'objet Batch métier vers JPA et inversement 6. /infrastructure/persistence/entities/Batch.java Entité JPA "Batch" 7. /infrastructure/persistence/repositories/BatchRepository.java Implémentation JPA de la persistance des données 8. /infrastructure/persistence/db/AuditTrail.java Traçabilité des modifications 9. Table de BDD batch

4.5. Autres endpoints

Entité	Endpoints
Sécurité	POST /api/v1/login POST /api/v1/logout
Utilisateurs	GET /api/v1/users POST /api/v1/users/search POST /api/v1/users GET /api/v1/users/{id} PUT /api/v1/users/{id} DELETE /api/v1/users/{id}
Traçabilité	GET /api/v1/audit_trail POST /api/v1/audit_trail/search
Fournisseurs	GET /api/v1/suppliers POST /api/v1/suppliers/search POST /api/v1/suppliers GET /api/v1/suppliers/{id} PUT /api/v1/suppliers/{id} DELETE /api/v1/suppliers/{id}
Composants	GET /api/v1/components POST /api/v1/components/search POST /api/v1/components GET /api/v1/components/{id} PUT /api/v1/components/{id} DELETE /api/v1/components/{id} GET /api/v1/components/{componentId}/controls POST /api/v1/components/{componentId}/controls PUT /api/v1/components/{componentId}/controls/{controlId} DELETE /api/v1/components/{componentId}/controls/{controlId}
Lots	GET /api/v1/batches POST /api/v1/batches/search POST /api/v1/batches GET /api/v1/batches/{batchId} PUT /api/v1/batches/{batchId} DELETE /api/v1/batches/{batchId} GET /api/v1/batches/{batchId}/analysis POST /api/v1/batches/{batchId}/analysis PUT /api/v1/batches/{batchId}/analysis/{analyseId} DELETE /api/v1/batches/{batchId}/analysis/{analyseId} GET /api/v1/batches/{batchId}/deviations POST /api/v1/batches/{batchId}/deviations PUT /api/v1/batches/{batchId}/deviations/{deviationId} DELETE /api/v1/batches/{batchId}/deviations/{deviationId}
Commandes	GET /api/v1/orders POST /api/v1/orders/search POST /api/v1/orders GET /api/v1/orders/{id} PUT /api/v1/orders/{id} DELETE /api/v1/orders/{id}

5. Regroupement des Classes (API / Back-end)



Annexes

I. Terminologie

Concepts technique

API	Interface logicielle permettant à deux applications de communiquer entre elles.
BDD	Base de données stockant et structurant les informations du système.
HTTP	Protocole standard de transfert de données sur le web.
MPD	Représentation textuelle et graphique des tables de la BDD.
Framework	Ensemble d'outils structurant le développement de l'application (ex: Spring).
API RESTful	Interface d'échange web respectant strictement les standards HTTP
SPA	Single Page Application : Application web fluide dont le contenu se charge sans recharger la page (ex: Angular).
IHM	Interface visuelle permettant à l'utilisateur de piloter l'application.
Adapter	Composant d'infrastructure traduisant les signaux extérieurs pour le domaine.
SQL	Structured Query Language : Langage standardisé pour interroger et manipuler la base de données.
Cache	Mémoire temporaire ultra-rapide limitant les accès BDD (ex: Redis).
Container	Environnement isolé embarquant l'application et ses dépendances (ex: Docker).
Back-end	Partie serveur de l'application gérant la logique métier et les données.
Front-end	Partie interface utilisateur s'exécutant dans le navigateur web.
CI/CD	Automatisation de l'intégration, des tests et du déploiement continu.
JDBC	API Java standard pour se connecter aux bases de données.
RESP	Protocole de communication interne, simple et rapide, utilisé par Redis.

Architecture

DDD	Domain Driven Design : Architecture guidée et centrée sur le modèle métier.
Domaine	Cœur de l'application contenant les règles métier pures.
Application	Couche orchestrant les cas d'utilisation et les flux.
Infrastructure	Couche gérant les détails techniques (BDD, API, sécurité).
Agrégat	Ensemble d'objets métier liés, manipulés comme une unité.
Entité	Objet métier défini par une identité unique et persistante.
DTO	Objet de transfert de données entre l'API et l'application.
Mapper	Composant convertissant les données d'une couche à l'autre.
Repository	Interface définissant le contrat d'accès aux données persistantes.

Sécurité & Infrastructure

Opaque Token	Jeton aléatoire unique servant de clé de session technique.
Redis	Base de données en mémoire stockant les sessions utilisateurs et le cache.
Spring Security	Composant du framework gérant l'authentification et les habilitations d'accès.
Audit Trail	Historique immuable des actions critiques pour la traçabilité.
JPA / Hibernate	Composant du framework gérant la persistance des objets Java en tables SQL.
Endpoint	URL d'API exposant un point d'accès technique précis.
REST	REpresentational State Transfer : Standard pour les services web HTTP.